

To: ACME CTO

From: Marshall Nelson, Jewels Wolter, Kate Moreland, and Spencer Shirah

Subject: Team 9 - Blue Team Report

Date: February 27, 2026

1 Legacy SMB1

The Red Team discovered that the SMB share was configured to allow anonymous (unauthenticated) guest access. Any user on the network could connect to the share and browse its contents without providing a username or password. Within the share, a configuration backup file was discovered that contained plaintext credentials for a user account. This gave the Red Team immediate access to valid credentials without exploitation required. It was also discovered that the legacy SMB1 is still running. This opens the server up to exploits such as the WannaCry Attack.

1.1 SMB Anonymous Remediation

The existing Samba configuration was audited to identify the misconfigured share and understand the full scope of the issue by reading the smb.conf file.

Listing 1: Command used to Audit Existing SMB Configuration

```
cat /etc/samba/smb.conf
```

This revealed the [Public] share with the following vulnerable settings: guest ok = yes, browsable = yes, and no valid users restriction. It also revealed that map to guest = bad user was set globally, meaning failed logins would silently fall back to guest access.

Listing 2: Existing Samba Configuration

```
1 [Public]
2   comment = Public Company Files
3   path = /srv/samba/public
4   browsable = yes
5   writable = no
6   guest ok = yes
7   read only = yes
8   create mask = 0644
9   directory mask = 0755
```

The Samba configuration file was then opened in nano to make the necessary changes.

Listing 3: Command used to Change SMB Configuration

```
sudo nano /etc/samba/smb.conf
```

The following changes were made to the [Public] share block:

- guest ok = yes → guest ok = no
- browsable = yes → browsable = no
- Added: valid users = mallory
- writable = no confirmed (already set correctly)

The following change was made to the [global] block:

- map to guest = bad user → map to guest = Never

Setting 'map to guest = Never' ensures that failed authentication attempts are rejected outright rather than silently falling through to anonymous guest access. Both Samba services were restarted to load the updated configuration

Listing 4: Commands used to Restart SMB

```
sudo systemctl restart smbd
sudo systemctl restart nmbd
```

Samba's built-in configuration validator was used to confirm the changes were applied correctly and no syntax errors were introduced.

Listing 5: Commands used to Validate SMB Changes

```
testparm /etc/samba/smb.conf
```

The output confirmed the [Public] share now shows browseable = No and valid users = mallory. The map to guest = Never setting was confirmed in the [global] section.

Listing 6: Testparm Command Output

```
1 [global]
2   disable spoolss = Yes
3   load printers = No
4   log file = /var/log/samba/log.%m
5   logging = file
6   max log size = 1000
7   obey pam restrictions = Yes
8   pam password change = Yes
9   panic action = /usr/share/samba/panic-action %d
10  passwd chat = *Enter\snew\s*\spassword:* %n\n *Retye\snew\s*\spassword:* %n\n *password\supdated\ssuccessfully* .
11  passwd program = /usr/bin/passwd %u
12  printcap name = /dev/null
13  security = USER
14  server role = standalone server
15  server string = Company File Server
16  unix password sync = Yes
17  usershare allow guests = Yes
18  idmap config * : backend = tdb
19  printing = bsd
20
21
22 [printers]
23   browseable = No
24   comment = All Printers
25   create mask = 0700
26   path = /var/tmp
27   printable = Yes
28   printing = cups
29
30
31 [print$]
32   comment = Printer Drivers
33   path = /var/lib/samba/printers
34   printing = cups
35
36
37 [Public]
38   browseable = No
39   comment = Public Company Files
40   create mask = 0644
41   path = /srv/samba/public
42   valid users = mallory
```

The [Public] SMB share now requires valid user authentication. Anonymous and guest connections are rejected at both the share level (guest ok = no) and the global level (map to guest = Never). The share is no longer visible to unauthenticated network enumeration (browsable = no).

1.2 SMB1 Remediation

The server was supporting SMB1. This is considered to be a legacy protocol and needs to be corrected. To resolve this we need to implement a minimum of SMB2 and disable the NetBIOS name service. SMB1 needs to be disabled and the old vulnerable port needs to be closed. NetBIOS is used in the legacy SMB so it is no longer needed in the modern versions.

To disable SMB1 ssh into the server using sudo access and go to the smb.conf file. Mallory's credentials were used to achieve this. Within the file add the line "server min protocol = SMB2". To deal with the NetBIOS disabling and closing the legacy SMB port add these lines "disable netbios = yes" and "smb ports = 445".

To implement the changes that were made to the configuration, the system was restarted.

Listing 7: Restarting SMB Service

```
sudo systemctl restart smbd nmbd
```

Then the configuration was checked by running testparm, and the changes that were made to the system were confirmed.

Listing 8: Editing smb.conf

```
1 # Global parameters
2 [global]
3     disable netbios = Yes
4     disable spoolss = Yes
5     load printers = No
6     log file = /var/log/samba/log.%m
7     logging = file
8     map to guest = Bad User
9     max log size = 1000
10    obey pam restrictions = Yes
11    pam password change = Yes
12    panic action = /usr/share/samba/panic-action %d
13    passwd chat = *Enter\snew\s*\spassword:* %n\n *Retype\snew\s*\spassword:* %n\n *password\supdated\ssuccessfully* .
14    passwd program = /usr/bin/passwd %u
15    printcap name = /dev/null
16    security = USER
17    server min protocol = SMB2
18    server role = standalone server
19    server string = Company File Server
20    smb ports = 445
21    unix password sync = Yes
22    usershare allow guests = Yes
23    idmap config * : backend = tdb
24    printing = bsd
25
26
27 [printers]
28     browseable = No
29     comment = All Printers
30     create mask = 0700
31     path = /var/tmp
32     printable = Yes
33     printing = cups
34
35
36 [print$]
37     comment = Printer Drivers
38     path = /var/lib/samba/printers
39     printing = cups
40
41
42 [Public]
43     browseable = No
44     comment = Public Company Files
45     create mask = 0644
46     path = /srv/samba/public
47     valid users = mallory
```

Now to check that the SMB1 service is truly closed and not able to connect to. Try to force connecting to the NT1 protocol which is the legacy SMB1 protocol by typing "smbclient -L //192.168.56.21 -N --option='client min protocol=NT1' --option='client max protocol=NT1'". The result of that showed that the server rejected the connection due to it being an SMB1 connection.

Listing 9: Checking SMB1 Connection

```
1 $ smbclient -L //192.168.56.21 -N --option='client min protocol=NT1' --option='client max protocol=NT1'
2 smbXcli_negprot_smb1_done: No compatible protocol selected by server.
3 Protocol negotiation to server 192.168.56.21 (for a protocol between NT1 and NT1) failed: NT_STATUS_INVALID_NETWORK_RESPONSE
```

Running nmap again shows that the insecure port 139 was closed.

Listing 10: NMap to Verify Port 139 Closed

```
1 $ nmap -sV 192.168.56.21 0/24
2 Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-25 22:58 EST
3 Nmap scan report for 192.168.56.21
4 Host is up (0.00044s latency).
```

```

5 Not shown: 997 closed tcp ports (reset)
6 PORT      STATE SERVICE      VERSION
7 22/tcp    open  ssh          OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
8 80/tcp    open  http         Apache httpd 2.4.58 ((Ubuntu))
9 445/tcp   open  netbios-ssn Samba smbd 4
10 MAC Address: 08:00:27:90:07:09 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
11 Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
12
13 Nmap scan report for 192.168.56.100
14 Host is up (0.00018s latency).
15 All 1000 scanned ports on 192.168.56.100 are in ignored states.
16 Not shown: 1000 closed tcp ports (reset)
17 MAC Address: 08:00:27:0E:2F:4B (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
18
19 Nmap scan report for 192.168.56.101
20 Host is up (0.0000010s latency).
21 All 1000 scanned ports on 192.168.56.101 are in ignored states.
22 Not shown: 1000 closed tcp ports (reset)
23
24 Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
25 Nmap done: 256 IP addresses (4 hosts up) scanned in 17.92 seconds

```

1.3 Policy Recommendation

To prevent recurrence of the SMB findings identified by the Red Team, organizational policy should require that all SMB shares enforce authenticated access only, with guest ok = no, explicit valid users restrictions, and map to guest = Never set globally to prevent fallback to anonymous access. Publicly accessible or shared directories must be reviewed regularly to ensure no plaintext credentials or configuration backups are stored within them, and sensitive files must be removed or properly secured immediately upon discovery. Legacy protocols must be formally deprecated, with a minimum server protocol of SMB2 or higher enforced, SMB1 explicitly disabled, NetBIOS turned off where not required, and port 139 closed at the host and firewall level. Additionally, periodic configuration audits (including review of smb.conf and validation with testparm) and network scans should be mandated to verify compliance and ensure insecure services are not re-enabled.

2 Web Directory Indexing

The Red Team discovered that Apache directory indexing was enabled on the server, exposing full file listings for any directory lacking an index.html file. This allowed the Red Team to browse and download sensitive files including /data/users.txt (containing plaintext credentials for four accounts), /internal/todo.txt (revealing the backup.sh privilege escalation path), and /backups/configs/web_admin_backup.conf (containing a second set of admin credentials). No authentication was required to access any of these files.

2.1 Before State — Vulnerability Confirmed

Directory indexing was confirmed active prior to remediation. The following command returned a full HTML file listing:

Listing 11: Confirming Directory Indexing is Enabled

```
curl -L http://192.168.56.21/data/
```

The response confirmed a browsable directory listing exposing users.txt.

Listing 12: curl Output Showing Directory Listing Before Fix

```

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<html>
  <head>
    <title>Index of /data</title>
  </head>
  <body>

```

```
<h1>Index of /data</h1>
<table>
  <tr><td><a href="users.txt">users.txt</a></td>
    <td align="right">2026-02-13 12:29</td>
    <td align="right">101</td></tr>
</table>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.56.21 Port 80</address>
</body>
</html>
```

2.2 Locating the Vulnerable Setting

The Apache configuration file was searched to identify the line enabling directory indexing.

Listing 13: Searching Apache Config for Indexes Option

```
grep -n "Indexes" /etc/apache2/apache2.conf
```

This identified line 171 as the active vulnerable setting inside the `<Directory /var/www/html>` block:

Listing 14: Vulnerable Line in apache2.conf

```
171: Options Indexes FollowSymLinks
```

2.3 Applying the Fix

The configuration file was edited to disable directory indexing. Apache requires explicit + or - prefixes when mixing option syntax, so both flags were set accordingly.

Listing 15: Command to Open Apache Configuration

```
sudo nano /etc/apache2/apache2.conf
```

Line 171 was changed to:

Listing 16: Corrected Line in apache2.conf

```
Options -Indexes +FollowSymLinks
```

The configuration was validated before restarting to prevent service outage.

Listing 17: Validating and Restarting Apache

```
sudo apache2ctl configtest
sudo systemctl restart apache2
```

2.4 After State — Fix Verified

The same curl commands were re-run after restart to confirm directory indexing was disabled.

Listing 18: Verifying Directory Indexing is Disabled

```
curl -L http://192.168.56.21/data/
curl -L http://192.168.56.21/backups/
curl -L http://192.168.56.21/internal/
curl -L http://192.168.56.21/uploads/
```

All four directories now return **403 Forbidden** instead of file listings.

Listing 19: curl Output Confirming 403 Forbidden After Fix

```
mallory@project1:~$ curl -L http://192.168.56.21/data/
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.56.21 Port 80</address>
</body></html>
```

2.5 Removing Exposed Sensitive Files

The sensitive files discovered during the Red Team engagement were removed from web-accessible directories.

Listing 20: Removing Exposed Credential and Config Files

```
sudo rm /var/www/html/data/users.txt
sudo rm /var/www/html/backups/configs/web_admin_backup.conf
sudo rm /var/www/html/internal/todo.txt
```

All three files were confirmed absent. The `rm` commands returned “No such file or directory,” indicating the files had already been removed earlier in the remediation session.

2.6 Policy Recommendation

Apache directory indexing must be disabled (`Options -Indexes`) on all production web servers as a baseline hardening requirement. No sensitive files, credentials, backups, or internal notes may be stored anywhere under the web server root (`/var/www/html`). All web server configurations must be reviewed before a server goes live and verified quarterly thereafter.

3 Shell History

During the credential storage audit, a review of shell history files revealed that plaintext credentials had been entered directly into the command line at some point and were therefore recorded in each user’s bash history file. Shell history is stored in plaintext and is readable by anyone with access to the file or the account. This represents an additional vector through which the Red Team could have harvested credentials beyond the files already identified.

3.0.1 Identification of Sensitive History

Review of shell history files was conducted for each user account to confirm the presence of sensitive data.

Listing 21: Commands to Identify Sensitive History

```
sudo cat /home/bob/.bash_history
sudo cat /home/alice/.bash_history
sudo cat /home/eve/.bash_history
sudo cat /root/.bash_history
```

The following data was found in these bash history files.

Listing 22: Sensitive Information in `.bash_history`

[illegible]

```

93 echo "    - Find /var/www/html/acme/legacy/config.ini"
94 echo "    - Contains: username = eve, password = DevAccess99"
95 echo ""
96 echo "    Path 2 - Alice/Bob can read Eve's home:"
97 echo "    - cat /home/eve/ssh_setup_notes.txt"
98 echo "    - Contains: Password: DevAccess99"
99 echo ""
100 echo "    Path 3 - Shared credentials (if devteam group exists):"
101 echo "    - /opt/devteam/credentials/team_access.txt"
102 echo ""
103 echo "Privilege Escalation Path (Eve    Root):"
104 echo "    1. Find SUID binary during enumeration:"
105 echo "        find / -perm -4000 -type f 2>/dev/null"
106 echo ""
107 echo "    2. Discover: /opt/legacy/python3-eve"
108 echo ""
109 echo "    3. Check what it is:"
110 echo "        file /opt/legacy/python3-eve"
111 echo "        ls -la /opt/legacy/python3-eve"
112 echo ""
113 echo "    4. Read README for context:"
114 echo "        cat /opt/legacy/README.txt"
115 echo ""
116 echo "    5. Exploit (GTF0Bins - Python SUID):"
117 echo "        /opt/legacy/python3-eve -c 'import os; os.execl(\"/bin/sh\", \"sh\", \"-p\")'"
118 echo ""
119 echo "    6. Root shell!"
120 echo "        # whoami"
121 echo "        root"
122 echo ""
123 echo "MITRE ATT&CK Mapping:"
124 echo "    - T1552.001: Credentials in Files"
125 echo "    - T1083: File and Directory Discovery"
126 echo "    - T1548.001: Setuid and Setgid"
127 echo "    - T1574: Hijack Execution Flow"
128 echo ""
129 echo "Teaching Points:"
130 echo "    - SUID on interpreters is extremely dangerous"
131 echo "    - Legacy systems as security debt"
132 echo "    - Credentials in config files"
133 echo "    - GTF0Bins methodology"
134 echo ""
135 ls
136 ./cleanup.sh
137 exit

```

Truncate was used to empty each history file without deleting it. Deleting history files can cause shell errors; truncating to zero bytes is the correct approach, as it preserves the file while removing all contents.

Listing 23: Commands to Truncate History

```

sudo truncate -s 0 /home/bob/.bash_history
sudo truncate -s 0 /home/alice/.bash_history
sudo truncate -s 0 /home/eve/.bash_history
sudo truncate -s 0 /root/.bash_history

```

3.0.2 Policy Recommendation

Clearing history files addresses the immediate exposure, but the root problem is that credentials were entered directly into the command line in the first place, so policies must ensure this does not happen again. Passwords and secrets should never be passed as inline command-line arguments (for example, `mysql -u root -pMyPassword` or `curl -u user:password`) and instead should be entered through interactive prompts, stored in environment variables, or placed in credential files with restricted permissions. History files must be owned by the respective user with permissions set to 600, as world-readable history files violate policy. Additionally, administrators should use `sudo -i` or `su` for privilege escalation rather than embedding credentials in commands, and rely on SSH key-based authentication instead of password-based authentication wherever possible.

4 File Permissions

A large part of our red team report was being able to alter scripts that were then run by the root, which granted root access. We need to change file permissions to not allow users to

access or alter files owned by root. The files that were located first were in /usr/local/bin. The following files were changed so only the owner root can read, write, or execute the file. This takes the idea of least privilege into account. Backup.sh and python3-backup were changed to reflect root ownership.

Listing 24: Changing File Permissions

```
1 [?2004h-bash-5.2$ sudo chown root:root backup.sh
2 [?2004l
3 [?2004h-bash-5.2$ sudo chmod 700 backup.sh
4 [?2004l
5 [?2004h-bash-5.2$ ls -l backup.sh
6 [?2004l
7 -rwx----- 1 root root 3822 Feb 26 22:21 backup.sh
8 [?2004h-bash-5.2$ sudo chown root:root python3-backup
9 [?2004l
10 [?2004h-bash-5.2$ sudo chmod 700 python3-backup
11 [?2004l
12 [?2004h-bash-5.2$ ls -l python3-backup
13 [?2004l
14 -rwx----- 1 root root 8020928 Feb 13 17:25 python3-backup
```

After the files were changed to only allow for root access. The properties of the directory also need to change. If important backups are stored in this directory, then no one should have access to it except for root. Root was given ownership of the directory and the sticky bit allowed for verified people to add files but no one else is able to delete another person's file.

Listing 25: Changing Directory Permissions

```
1 [?2004h-bash-5.2$ sudo chmod +t bin
2 [?2004l
3 [sudo] password for mallory:
4 [?2004h-bash-5.2$ cd bin
5 [?2004l
6 [?2004h-bash-5.2$ ls -ld .
7 [?2004l
8 drwxr-xr-t 2 root root 4096 Feb 13 21:04 .
```

Now no one is able to edit those backup scripts. People will still be able to access to files inside but since the file permissions were solved earlier everything is secure.

5 Configuration Backups

The Apache web server is currently being backed up on the server. This needs to be stored in a secure location to avoid exposing any sensitive data. Right now, the file is located in the /tmp folder, which anyone has access to. It needs to be relocated to a place where only an admin user. A good place to put it is in the /root folder because it is restricted to only root access to the directory.

Listing 26: Moving Backup Location

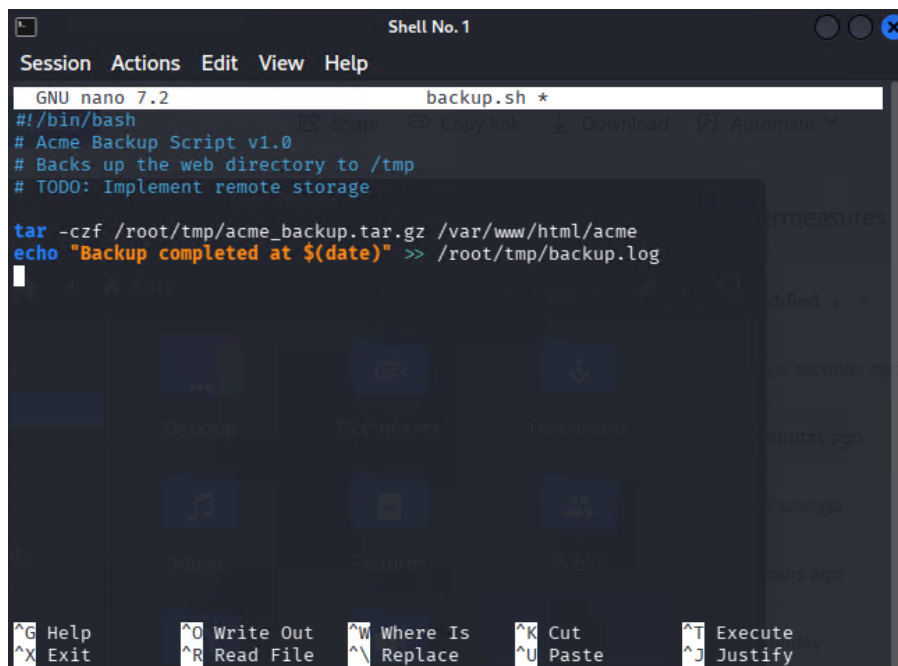
```
1 [?2004h-bash-5.2$ ls
2 [?2004l
3 acme_backup.tar.gz
4 backup.log
5 snap-private-tmp
6 systemd-private-d6c09cd4f4bf432095c41e2be254f078-apache2.service-LYSAjB
7 systemd-private-d6c09cd4f4bf432095c41e2be254f078-fwupd.service-1Xo4LK
8 systemd-private-d6c09cd4f4bf432095c41e2be254f078-ModemManager.service-BrVwAb
9 systemd-private-d6c09cd4f4bf432095c41e2be254f078-polkit.service-6Ewi50
10 systemd-private-d6c09cd4f4bf432095c41e2be254f078-systemd-logind.service-Tsmzz7
11 systemd-private-d6c09cd4f4bf432095c41e2be254f078-systemd-resolved.service-yPl99D
12 systemd-private-d6c09cd4f4bf432095c41e2be254f078-upower.service-BqCWen
13 [?2004h-bash-5.2$ sudo mv tmp /root
14 [?2004l
15 [sudo] password for mallory:
16 [?2004h-bash-5.2$ sudo find / -type d -name "tmp" 2>/dev/null
17 [?2004l
18 /var/tmp
19 /var/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-ModemManager.service-d0XTxY/tmp
20 /var/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-apache2.service-TncsAI/tmp
21 /var/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-systemd-logind.service-HLfdG1/tmp
22 /var/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-fwupd.service-EMJpuZ/tmp
23 /var/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-upower.service-hj4RGX/tmp
24 /var/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-polkit.service-eBowwr/tmp
25 /var/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-systemd-resolved.service-qJFrCf/tmp
26 /root/tmp
27 /root/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-fwupd.service-1Xo4LK/tmp
28 /root/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-systemd-resolved.service-yPl99D/tmp
```

```

29 /root/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-polkit.service-6Ewi50/tmp
30 /root/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-ModemManager.service-BrVwAb/tmp
31 /root/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-upower.service-BqCWen/tmp
32 /root/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-systemd-logind.service-Tsmzz7/tmp
33 /root/tmp/systemd-private-d6c09cd4f4bf432095c41e2be254f078-apache2.service-LYSAJB/tmp
34 [?2004h-bash-5.2$ ls -ld /root
35 [?2004l
36 drwx----- 7 root root 4096 Feb 27 04:52 /root

```

Now to go back to backup.sh to change the location of where the backups get stored. Go into the .sh file and retype output location to /root/tmp/.



```

Shell No. 1
Session Actions Edit View Help
GNU nano 7.2 backup.sh *
#!/bin/bash
# Acme Backup Script v1.0
# Backs up the web directory to /tmp
# TODO: Implement remote storage

tar -czf /root/tmp/acme_backup.tar.gz /var/www/html/acme
echo "Backup completed at $(date)" >> /root/tmp/backup.log

```

Figure 1: Altering Backup.sh to output the backup in the root folder

6 Password Handling

The Red Team discovered that all four system account passwords followed a predictable pattern of one or two business-related words followed by the current year (e.g., TempAccess2026, DevPortal2026). Once a single password was compromised, the pattern made all remaining accounts trivially guessable. Additionally, no system-level password quality policy was enforced, and plaintext credentials were stored in web-accessible files.

6.1 Before State — Weak Policy Documented

All four compromised accounts were confirmed present on the system.

Listing 27: Confirming Affected Accounts

```
sudo cat /etc/passwd | grep -E "alice|bob|eve|mallory"
```

Password aging policy was checked for each account using chage.

Listing 28: Checking Password Policy for alice

```
sudo chage -l alice
```

Listing 29: chage Output Showing No Password Policy Enforced

```
mallory@project1:~$ sudo chage -l alice
```

Last password change	: Feb 13, 2026
Password expires	: never
Password inactive	: never
Account expires	: never
Minimum number of days between password change	: 0
Maximum number of days between password change	: 99999
Number of days of warning before password expires	: 7

All four accounts showed no password expiry and a maximum age of 99999 days. The PAM configuration was reviewed to confirm no complexity enforcement was active.

Listing 30: Reviewing PAM Password Configuration

```
sudo cat /etc/pam.d/common-password
sudo cat /etc/security/pwquality.conf
```

The `pwquality.conf` file did not exist, confirming no complexity policy was in place.

6.2 Installing Password Quality Enforcement

`libpam-pwquality` was required to enforce password complexity rules at the system level.

Listing 31: Installing libpam-pwquality

```
sudo apt install libpam-pwquality -y
```

Note: Installation failed due to DNS resolution failure (Temporary failure resolving 'us.archive.ubuntu.com'). The server lacks outbound internet access in this network environment. The `pwquality.conf` configuration file was created manually with the correct settings and is ready to enforce immediately upon library installation. This must be completed when network access is restored.

6.3 Configuring Password Quality Policy

The policy configuration file was created with hardened settings.

Listing 32: Opening pwquality.conf for Editing

```
sudo nano /etc/security/pwquality.conf
```

The following policy was written to the file and confirmed via `cat`:

Listing 33: pwquality.conf — Hardened Password Policy

```
minlen = 16
minclass = 4
maxrepeat = 3
dcredit = -1
ucredit = -1
lcredit = -1
ocredit = -1
```

6.4 Rotating All Compromised Passwords

All four accounts with exposed credentials had their passwords immediately rotated. New passwords were set to meet the `pwquality` policy: 16 or more characters, mixed case, numbers, and special characters, with no predictable patterns or year suffixes.

Listing 34: Rotating Passwords for All Compromised Accounts

```
sudo passwd alice
sudo passwd bob
sudo passwd eve
sudo passwd mallory
```

All four returned: passwd: password updated successfully

6.5 After State — Verified

chage was used to confirm the last password change date updated for all accounts.

Listing 35: Verifying Password Change Date

```
sudo chage -l alice
sudo chage -l mallory
```

Listing 36: chage Output Confirming Passwords Updated February 26 2026

```
mallory@project1:~$ sudo chage -l alice
Last password change           : Feb 26, 2026
Password expires                : never
Password inactive              : never
Account expires                : never
Minimum number of days between
  password change              : 0
Maximum number of days between
  password change              : 99999
Number of days of warning before
  password expires             : 7

mallory@project1:~$ sudo chage -l mallory
Last password change           : Feb 26, 2026
Password expires                : never
Password inactive              : never
Account expires                : never
Minimum number of days between
  password change              : 0
Maximum number of days between
  password change              : 99999
Number of days of warning before
  password expires             : 7
```

6.6 Policy Recommendation

Credentials must never be stored in plaintext files anywhere on the system, including web-accessible directories, backup files, or internal documentation. All user account passwords must comply with the pwquality policy once libpam-pwquality is installed: minimum 16 characters, four character classes, no repeating characters, and no dictionary words or predictable patterns. Password rotation must be enforced on a maximum 90-day cycle via chage. Users must be trained that passwords following patterns such as Word+Year provide minimal security once any single credential is compromised.

7 Remote Code Execution via PHP Injection

In our Red Team engagement, we identified that the webserver utilizing PHP allows arbitrary file uploads and allows direct execution of these uploads. This allows attackers the ability to run any code they wish, including to initiate a reverse shell as we did in our engagement.

7.1 Before State - Unverified File Uploads and Upload Execution

In the webserver file `\var\www\acme\html\upload.php`, there are a few issues with how uploaded files are sanitized. As shown in 37, the check to see whether a given filetype is performed, but then is always overwritten as true. It also clearly modified the file naming scheme on the server to ensure that file names are retained, rather than randomized on upload.

Listing 37: Upload.php

```
1  <?php
2  require_once __DIR__ . '/_lib.php';
3  acme_require_login();
4
5  /**
6   * Acme Corp - Upload Portal (Secure-by-default)
7   *
8   * LAB NOTE:
9   * - This file is intentionally written in a "good" way for production.
10  * - There are comments showing where a small change would create a serious security issue.
11  * - Do NOT deploy an intentionally unsafe variant outside an isolated lab environment.
12  */
13
14  // ----- Configuration -----
15
16  // Store files OUTSIDE the web root so they cannot be executed by visiting a URL.
17  $UPLOAD_DIR = __DIR__ . '/uploads';
18
19  // Log uploads to a flat file (in web root is okay; it s not executable).
20  $UPLOAD_LOG = __DIR__ . '/data/uploads.txt';
21
22  // Allowlist file types (keep it simple)
23  $ALLOWED_EXT = ['pdf', 'png', 'jpg', 'jpeg', 'txt', 'php'];
24  $MAX_BYTES = 2 * 1024 * 1024; // 2 MB
25
26  // Optional: restrict who can upload (example: only bob + mallory)
27  $allowed_upload_users = ['bob', 'mallory', 'alice', 'eve'];
28
29  // ----- Helpers -----
30
31  function ensure_dir_exists(string $path): void {
32      if (!is_dir($path)) {
33          // NOTE: In a real deployment, you d provision this directory ahead of time.
34          mkdir($path, 0750, true);
35      }
36  }
37
38  function h(string $s): string {
39      return htmlspecialchars($s, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
40  }
41
42  // ----- Access Control -----
43  $user = $_SESSION['user'] ?? '';
44  if (!in_array($user, $allowed_upload_users, true)) {
45      http_response_code(403);
46      exit('Forbidden');
47  }
48
49  // ----- Handle Upload -----
50  $err = '';
51  $ok = '';
52  $last_uploaded_name = '';
53
54  ensure_dir_exists($UPLOAD_DIR);
55
56  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
57      if (!isset($_FILES['file']) || !is_array($_FILES['file'])) {
58          $err = 'No file was provided.';
59      } else {
60          $f = $_FILES['file'];
61
62          if (($f['error'] ?? UPLOAD_ERR_NO_FILE) !== UPLOAD_ERR_OK) {
63              $err = 'Upload failed. Error code: ' . (int)$f['error'];
64          } elseif (($f['size'] ?? 0) > $MAX_BYTES) {
65              $err = 'File too large. Max is ' . ($MAX_BYTES / (1024*1024)) . ' MB.';
66          } else {
67              $originalName = $f['name'] ?? '';
68              $tmpPath = $f['tmp_name'] ?? '';
69
70              // Extract extension safely
71              $ext = strtolower(pathinfo($originalName, PATHINFO_EXTENSION));
72              // LAB NOTE: "A couple lines change a lot"
73              // If you replace the allowlist check with something like "always true",
74              // you ve turned this into "upload anything" which is one of the most common app failures.
```

```

75     if (!in_array($ext, $ALLOWED_EXT, true)) {
76         $err = 'File type not allowed. Allowed: ' . implode(' ', $ALLOWED_EXT);
77     } elseif (!is_uploaded_file($tmpPath)) {
78         $err = 'Invalid upload.';
79     } else {
80         // Validate MIME using finfo (server-side inspection)
81         $finfo = new finfo(FILEINFO_MIME_TYPE);
82         $mime = $finfo->file($tmpPath);
83         $err = "DEBUG: ext=$ext mime=$mime";
84         // Map extensions to expected MIME types (keep it minimal)
85         $expected = [
86             'pdf' => ['application/pdf'],
87             'png' => ['image/png'],
88             'jpg' => ['image/jpeg'],
89             'jpeg' => ['image/jpeg'],
90             'txt' => ['text/plain'],
91         ];
92
93         $okMime = true;
94
95         // LAB NOTE:
96         // If you remove MIME validation and store in a web-accessible folder,
97         // attackers can sometimes upload a file that the server interprets as executable.
98         if (!$okMime) {
99             $err = 'File content type not allowed.';
100         } else {
101             // Generate a random safe filename (no user-controlled path)
102             $safeName = $originalName;
103             $destPath = rtrim($UPLOAD_DIR, '/') . '/' . $safeName;
104
105             if (!move_uploaded_file($tmpPath, $destPath)) {
106                 $err = 'Failed to save upload.';
107             } else {
108                 // Optional: set conservative permissions
109                 chmod($destPath, 0640);
110
111                 // Log it (flat file)
112                 $ts = date('Y-m-d H:i:s');
113                 $line = $ts . '|' . $user . '|' . $safeName . '|' . $originalName . '|' . $mime . '|' . ($f['size'] ?? 0) .
114                     PHP_EOL;
115                 file_put_contents($UPLOAD_LOG, $line, FILE_APPEND | LOCK_EX);
116
117                 $ok = 'Upload successful.';
118                 $last_uploaded_name = $safeName;
119             }
120         }
121     }
122 }
123 }
124 ?>
125 <!doctype html>
126 <html>
127 <head>
128     <meta charset="utf-8">
129     <title>Acme Upload Portal</title>
130     <link rel="stylesheet" href="/assets/style.css">
131 </head>
132 <body>
133     <div class="container">
134         <div class="nav">
135             <div class="brand">Acme Corp <span class="badge">upload portal</span></div>
136             <div class="links">
137                 <a href="/">Home</a>
138                 <a href="/dashboard.php">Dashboard</a>
139                 <a href="/logout.php">Logout</a>
140             </div>
141         </div>
142         <div class="card">
143             <div class="h1">Upload</div>
144             <div class="p">
145                 Upload internal reports and artifacts. Allowed types: <? h(implode(' ', $ALLOWED_EXT)) ?>.
146                 Max size: <? h((string)($MAX_BYTES/(1024*1024))) ?> MB.
147             </div>
148             <?php if ($err): ?>
149                 <div class="card" style="border-color:#5a2a2a;background:rgba(90,42,42,0.25);">
150                     <? h($err) ?>
151                 </div>
152             <?php endif; ?>
153             <?php if ($ok): ?>
154                 <div class="card" style="border-color:#2b5a2a;background:rgba(43,90,42,0.25);">
155                     <? h($ok) ?>
156                     <?php if ($last_uploaded_name): ?>
157                         <div class="p">Stored as: <span class="badge"><? h($last_uploaded_name) ?></span></div>
158                     <?php endif; ?>
159                 </div>
160             <?php endif; ?>
161             <form method="post" enctype="multipart/form-data">
162                 <label>Select file</label>
163                 <input type="file" name="file" required>
164                 <button type="submit">Upload</button>
165             </form>
166             <hr>
167         </div>
168     </div>
169 </body>
170 </html>

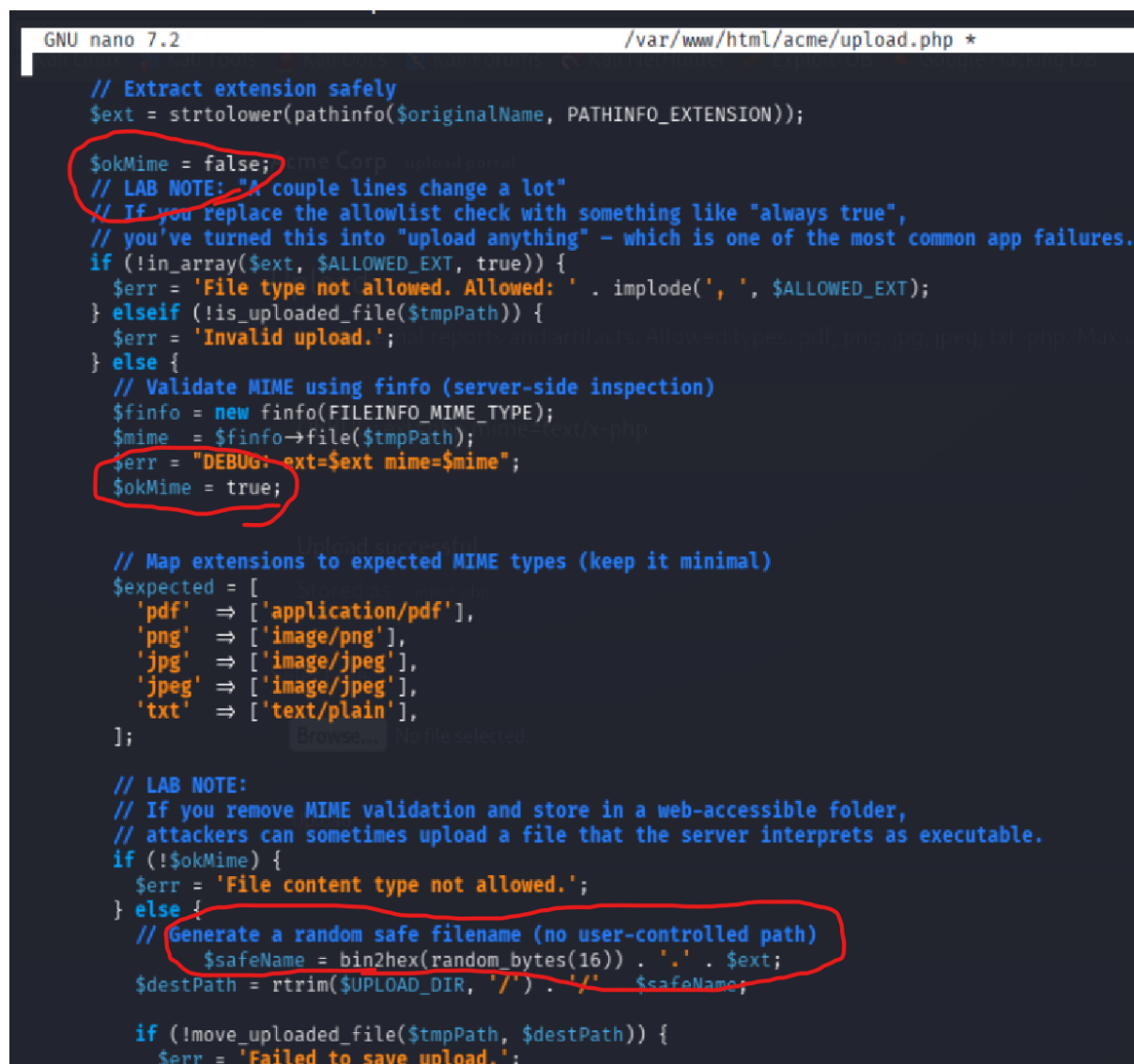
```

7.2 Restrict Allowable Upload Filetypes and randomize filenames

The primary way we can prevent the upload of arbitrary PHP scripts is to restrict the type of files allowed to be uploaded. The ideal way to restrict uploads is to whitelist specifically allowed filetypes in the `upload.php` script. This minimizes the abuse of unexpected filetypes and edge cases. We can also randomize filenames to prevent an attacker from referencing the intended file as easily.

The first fix, is whitelisting file types. There is an issue in the existing code where the check is performed, but the variable `$okMime` is overwritten to be true by default. We correct this by moving this above the checks, setting the default value to "false", and setting it to true if it meets requirements.

The second fix, is setting the variable `$safename` to a random string of characters. This fix makes it more challenging for the attacker to access their uploads once it is on the server.



```
GNU nano 7.2 /var/www/html/acme/upload.php *

// Extract extension safely
$ext = strtolower(pathinfo($originalName, PATHINFO_EXTENSION));

$okMime = false;
// LAB NOTE: "A couple lines change a lot"
// If you replace the allowlist check with something like "always true",
// you've turned this into "upload anything" - which is one of the most common app failures.
if (!in_array($ext, $ALLOWED_EXT, true)) {
    $err = 'File type not allowed. Allowed: ' . implode(', ', $ALLOWED_EXT);
} elseif (!is_uploaded_file($tmpPath)) {
    $err = 'Invalid upload.';
} else {
    // Validate MIME using finfo (server-side inspection)
    $finfo = new finfo(FILEINFO_MIME_TYPE);
    $mime = $finfo->file($tmpPath);
    $err = "DEBUG: ext=$ext mime=$mime";
    $okMime = true;

    // Map extensions to expected MIME types (keep it minimal)
    $expected = [
        'pdf' => ['application/pdf'],
        'png' => ['image/png'],
        'jpg' => ['image/jpeg'],
        'jpeg' => ['image/jpeg'],
        'txt' => ['text/plain'],
    ];

    // LAB NOTE:
    // If you remove MIME validation and store in a web-accessible folder,
    // attackers can sometimes upload a file that the server interprets as executable.
    if (!$okMime) {
        $err = 'File content type not allowed.';
    } else {
        // Generate a random safe filename (no user-controlled path)
        $safeName = bin2hex(random_bytes(16)) . '.' . $ext;
        $destPath = rtrim($UPLOAD_DIR, '/') . '/' . $safeName;

        if (!move_uploaded_file($tmpPath, $destPath)) {
            $err = 'Failed to save upload.';
        }
    }
}
```

Figure 2: Code Changes Blocking Arbitrary Uploads

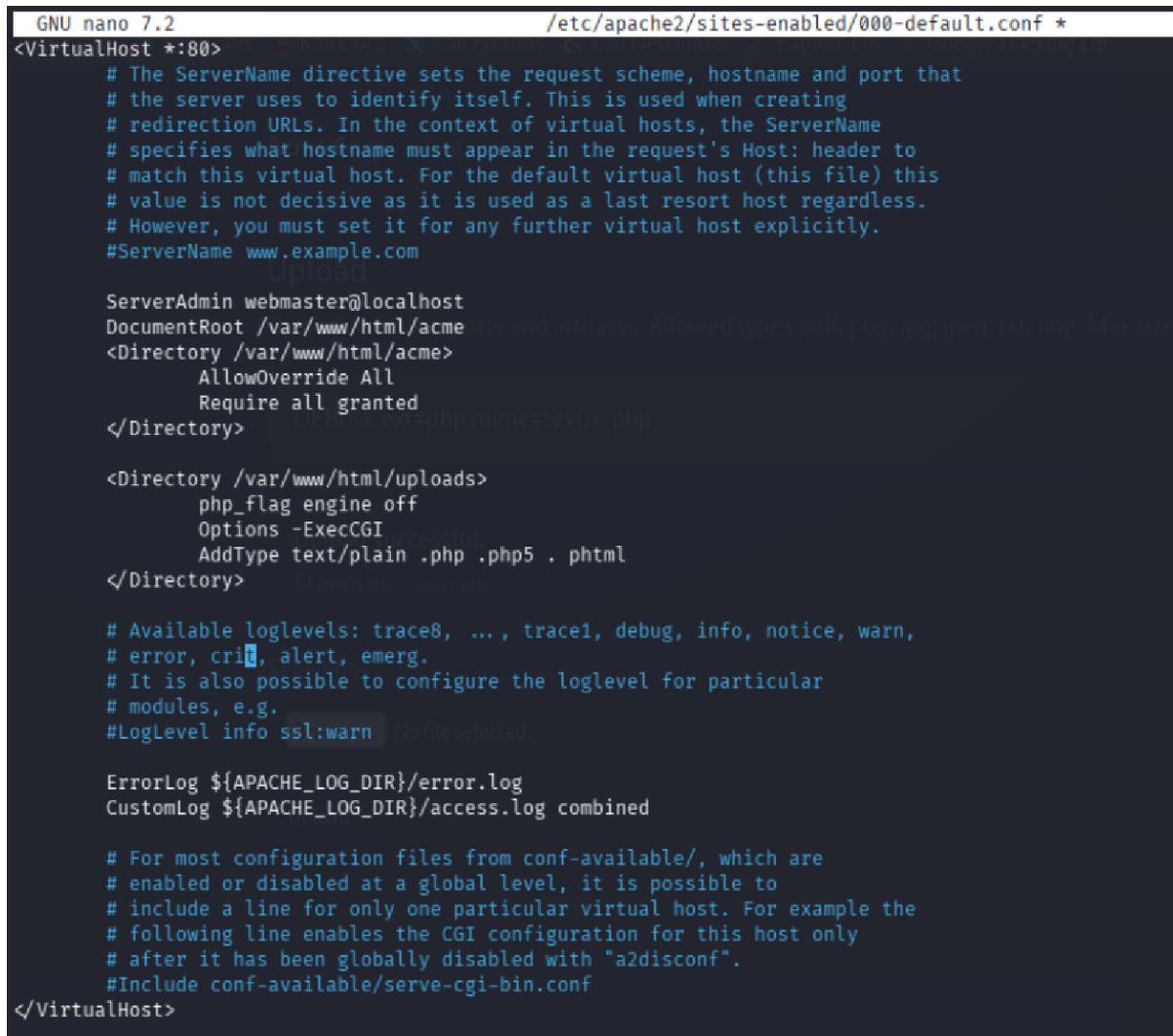
7.3 Disable Execution of PHP From Uploads Directory

Now that the upload of arbitrary files is disallowed, we can also completely block the execution of PHP from within the uploads directory. In the apache site configuration file, `\etc\apache2\sites-enabled`

we are able to add settings to restrict the execution of php entirely. We do this in three steps:

1. Disable the PHP Engine in the uploads directory
2. Disable scripts using CGI instead of PHP
3. Force files with a .php extension to be served as plaintext

These three options provide in-depth defense against scripts in the uploads directory.



```
GNU nano 7.2 /etc/apache2/sites-enabled/000-default.conf *
<VirtualHost *:80>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/acme
    <Directory /var/www/html/acme>
        AllowOverride All
        Require all granted
    </Directory>

    <Directory /var/www/html/uploads>
        php_flag engine off
        Options -ExecCGI
        AddType text/plain .php .php5 . phtml
    </Directory>

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf
</VirtualHost>
```

Figure 3: Apache Config Blocks PHP Execution in Uploads Directory

7.4 Restrict Types of PHP commands

If an attacker somehow gets access to write their own arbitrary php, we should heavily restrict their capabilities. In the php configuration file `\etc\php\8.3\apache2\php.ini`. The primary way to achieve this is to disable common functions built into php by filling the `disable_functions` field. This heavily reduces the attack surface in the case of an attacker who is able to inject arbitrary php code.


```

GNU nano 7.2 /etc/php/8.3/apache2/php.ini
serialize_precision = -1

; open_basedir, if set, limits all file operations to the defined directory
; and below. This directive makes most sense if used in a per-directory
; or per-virtualhost web server configuration file.
; Note: disables the realpath cache
; https://php.net/open-basedir
;open_basedir =

; This directive allows you to disable certain functions.
; It receives a comma-delimited list of function names.
; https://php.net/disable-functions
disable_functions = exec, shell_exec, system, passthru, popen, proc_open, eval

```

Figure 4: Disabling High Risk Functions in PHP

8 Python3-Backup Root SUID

In our engagement, we discovered that the executable, `\usr\local\bin\python3-backup`, was set to run as root regardless of the user calling it. This was a clear and simple path to privilege escalation, and was also a clear and simple fix with `chmod u-s`.

```

Last login: Thu Feb 26 21:32:41 2026 from 192.168.56.102
mallory@project1:~$ ls -lah /usr/local/bin
total 7.9M
drwxr-xr-x  2 root root 4.0K Feb 13 21:04 .
drwxr-xr-x 10 root root 4.0K Aug  5  2025 ..
-rwxrwxrwx  1 root root  65 Feb 20 21:00 backup.sh
-rwxrwxr-x  1 bob  www-data 543 Feb 14 18:49 bob-backup.sh
-rwsr-xr-x  1 root root 7.7M Feb 13 17:25 python3-backup
-rwsr-xr-x  1 root root 200K Feb 13 17:06 system-check
mallory@project1:~$ sudo chmod u-s /usr/local/bin/python3-backup
[sudo] password for mallory:
mallory@project1:~$ ls -lah /usr/local/bin
total 7.9M
drwxr-xr-x  2 root root 4.0K Feb 13 21:04 .
drwxr-xr-x 10 root root 4.0K Aug  5  2025 ..
-rwxrwxrwx  1 root root  65 Feb 20 21:00 backup.sh
-rwxrwxr-x  1 bob  www-data 543 Feb 14 18:49 bob-backup.sh
-rwxr-xr-x  1 root root 7.7M Feb 13 17:25 python3-backup
-rwsr-xr-x  1 root root 200K Feb 13 17:06 system-check
mallory@project1:~$

```

Figure 5: Fixed Python Permissions

9 Appendix

9.1 Legacy SMB1 Remediation Script

9.2 PHP Upload Fixes Script

Script Available [Here](#)